MEDIKIOSK CLINICAL PLATFORM DOCUMENTATION

PROJECT ANALYSIS SUMMARY

Project Analysis Summary

Last Analyzed: 2026-09-16 Commit:`f118085510bdf2fa44785ee3ca5aaf5b4875c209`

1. Executive Summary & Purpose

MediKiosk is an intelligent, touch- and voice-enabled clinical intake and triage kiosk platform engineered for high-density Outpatient Departments (OPDs) in Indian public hospitals. It includes specialized clinical modules for integrative medicine, combining Allopathic clinical triage with Ayurvedic *Dashavidha Pariksha* assessments.

The system streamlines clinical intake by empowering patients to self-record structured medical histories in English or Hindi prior to meeting their physician. It features a synchronous 15-rule deterministic emergency screening engine, automated prescription digitisation via Document OCR, an AYUSH Retrieval-Augmented Generation (RAG) engine over ancient classical texts, and a real-time doctor triage dashboard complete with full data provenance tracking and HL7 FHIR R4 export capability.

2. Architecture Overview

MediKiosk follows a multi-layer microservices and client-server architecture:

1. **Frontend Kiosk & Doctor Portal** (`frontend/src/`): Built with React 19, Vite 8, and TypeScript. Handles multi-lingual patient workflows (`/welcome`, `/notice`, `/consent`, `/chief-complaint`, `/interview`, `/ayush-assessment`, `/records-upload`, `/preview`) and clinician OPD queues (`/doctor/dashboard`, `/doctor/triage/:sessionId`). Offline capability is supported via IndexedDB (`idb-keyval`).
2. **Backend REST & WebSocket API** (`backend/src/`): Powered by Express 5 on Node.js 22. Manages session lifecycles, Zod schema validation, password hashing (`bcryptjs`), real-time safety evaluation (`redflags.ts`), AYUSH RAG retrieval (`ayushRag.ts`), and FHIR bundle generation (`fhir.ts`).
3. **AI & Speech Orchestration Router** (`backend/src/orchestration/router.ts`): A unified gateway coordinating AI model invocations across:
   - **Sarvam AI**: Saaras v3 ASR, Bulbul v3 TTS, Doc AI digitise OCR, and Mayura v1 Translation.
   - **Local MedGemma LLM**: `medgemma1.5:latest` executing via local Ollama instance for local red-flag checking and off-script Q&A.
   - **Google Gemini Cloud AI**: Gemini 1.5 Flash / 2.0 Flash for RAG synthesis, ICD-10 suggestions, entity extraction, and fallback reasoning.
4. **Data & Vector Storage Layer** (`Supabase PostgreSQL + pgvector`): Manages relational tables (`sessions`, `answers`, `ayush_assessments`, `medical_history`, `documents`, `summaries`, `consent_logs`, `audit_logs`) alongside 768-dimensional embeddings in `rag_chunks`. Real-time session updates are broadcast to the doctor portal via Supabase WebSocket publications.

3. Technology Stack Summary

Component Layer	Primary Technology	Auxiliary / Fallback Technology
Frontend Framework	React 19 + Vite 8 + TypeScript	TailwindCSS + Lucide Icons + Radix UI Slot
Client Storage	IndexedDB via `idb-keyval`	Web Speech API / Web Speech Synthesis
Backend Framework	Node.js 22 + Express 5 + TypeScript	Zod + `bcryptjs` + CORS + Multer
Database & Vector Store	Supabase PostgreSQL 15 + `pgvector`	Supabase Realtime WebSockets (`supabase_realtime`)
Vector Search / RAG	`pgvector` Similarity Search (`rag_chunks`)	Cosine similarity ranking (`retriever.ts`)
Speech-to-Text (ASR)	Sarvam AI Saaras v3	Web Speech API
Text-to-Speech (TTS)	Sarvam AI Bulbul v3	Web Speech Synthesis
Document OCR	Sarvam Doc AI digitise	Tesseract.js client-side WebAssembly
Clinical AI Engines	MedGemma 1.5 (Local Ollama)	Google Gemini 1.5 Flash / 2.0 Flash
Health Interoperability	HL7 FHIR R4 Bundle Generator (`fhir.ts`)	Simulated ABDM M1 / M2 / M3 Services

4. End-to-End Workflow Summary

CODE

```
[Patient Kiosk Intake]
   ├── 1. DPDP Notice & Consent (/notice, /consent)
   ├── 2. Chief Complaint Selection (/chief-complaint)
   ├── 3. Adaptive Interview (/interview) → Sarvam ASR / Voice-Match
   ├── 4. 15 Deterministic Red-Flag Rules (redflags.ts) → Auto-Flag Emergencies
   ├── 5. 12-Dimension AYUSH Assessment → Classical RAG Retrieval (ayushRag.ts)
   ├── 6. Prescription Document Upload → OCR & Gemini Entity Extraction
   └── 7. Submission → Generates Summary & Session Token
            |
            ▼ (Supabase Realtime WebSockets)
[Doctor Triage Dashboard]
   ├── 1. Live OPD Patient Queue with Red-Flag Indicators
   ├── 2. Provenance-Badged Clinical Summary ('patient_reported', 'ocr_extracted')
   ├── 3. Inline Fact Verification (flips status to 'doctor_entered')
   ├── 4. One-Click HL7 FHIR R4 Bundle Export
   └── 5. ABDM Care Context Linkage
```

5. Datasets Found in Repository

1. **OPD Clinical Dialogues Dataset** (`training/opd_dialogues.jsonl`): Synthetic doctor-patient OPD dialogues formatted for QLoRA fine-tuning.
2. **Ayurveda / Traditional Medicine to ICD-10 Bridge Dataset** (`rag_corpus/raw/ayurveda_icd/`): 3,002 entries (`AmareshHebbar/ayurveda-icd-sft`) mapping traditional Ayurvedic terms to ICD-10 CM diagnosis codes.
3. **Indian Vedas & Ayurveda Corpus** (`rag_corpus/raw/vedas/continueousPreTrainData.jsonl`): 39.51 MB pre-training dataset containing classical texts (Rig, Sama, Yajur, Atharva Vedas, Charaka Samhita, Sushruta Samhita, Rasa Jala Nidhi, and IRJAY articles).

6. Key Project Strengths

1. **Deterministic Red-Flag Safety Guarantee**: 15 deterministic red-flag rules execute synchronously prior to AI calls, preventing LLM hallucinations on emergency medical symptoms (chest pain, severe breathlessness, stroke signs, suicidal ideation).
2. **Strict Data Provenance**: Every clinical data point maintains a explicit provenance tag (`patient_reported`, `ocr_extracted`, `system_derived`, `doctor_entered`), ensuring absolute clinical accountability.
3. **Integrative Allopathic-Ayurvedic Clinical Workflow**: Seamlessly bridges Western OPD history taking with Ayurvedic *Dashavidha Pariksha* assessments, supported by vector-driven classical citation retrieval.
4. **Resilient Multimodal Failover**: High availability design with graceful degradation (Sarvam AI → MedGemma → Gemini → local rule-based fallbacks).
5. **Standards Compliance**: Built-in export of HL7 FHIR R4 bundles and ABDM M1/M2/M3 framework compatibility.